AutoFix Pro — Architecture Decision Records

Nombre(s): Hugo Arturo Villarreal Ortiz Enrique Canales Belmares Oscar Alexis Aldana Almanza	
Matrícula(s): 2142659 2142832 2144189	
Nombre del curso: Componentes de Lógica de Negocio	
Nombre del profesor: Miguel Ángel Gómez Marroquin	
Módulo / Actividad: Unidad 3 · AutoFix Pro	13/10/2026

Bibliografía / fuentes: Caso 06 AutoFix Pro.

Architecture Decision Records

ADR-001	Adoptar Domain Model (DDD) en lugar de Transaction Script o Table Module
Título	Adoptar Domain Model (DDD) en lugar de Transaction Script o Table Module
Estado	Aceptada
Contexto	AutoFix Pro gestiona el ciclo de vida de una OrdenServicio (Recibida, Diagnosticado, Cotizado, En reparación, Listo para entrega, Cerrada) con reglas interconectadas: rechazo parcial de Cotización con recálculo de montos (US-04), fallas adicionales que pausan el trabajo (US-08), reprogramación de FechaEntrega por demora de refacciones (US-11) y tarifas corporativas de flotilla (US-17). Las reglas cambian con frecuencia y varias deben cumplirse siempre, sin importar desde qué caso de uso se invoquen (por ejemplo: ningún trabajo inicia ni se cobra sin una Cotización con autorización válida).
Alternativas	**Transaction Script:** rápido de escribir, pero duplica reglas entre CU-01 a CU-05 y no protege invariantes; cada nueva política implica editar procedimientos largos. **Table Module:** adecuado para reportes y filtros sobre tablas (US-16), pero no controla el ciclo de vida por entidad ni sus invariantes. **Domain Model:** mayor esfuerzo inicial, pero concentra reglas e invariantes en OrdenServicio, Cotizacion, FechaEntrega y Kilometraje.
Decisión	Se adopta Domain Model como patrón principal para el núcleo transaccional del taller (OrdenServicio como Aggregate Root). Las consultas de lectura y los indicadores gerenciales (US-16) no usan el modelo de dominio; se resuelven con Queries (ADR-002).
Consecuencias	**Beneficios:** invariantes protegidas en un solo lugar, lenguaje ubicuo reflejado en el código, pruebas unitarias del dominio sin base de datos. **Costos aceptados:** curva de aprendizaje para el equipo, más clases y objetos de mapeo entre dominio y persistencia. **Riesgo:** sobrediseñar entidades para conceptos simples; se mitiga modelando como entidad solo lo que tiene identidad y ciclo de vida. **Trabajo posterior:** pruebas de las invariantes de OrdenServicio y revisión SOLID.
Trazabilidad	CU-01 a CU-05; US-01, US-04, US-08, US-10, US-11, US-16, US-17; RN-01, RN-02, RN-03, RN-04, RN-06, RN-09, RN-13. Evidencia: tabla comparativa de la Etapa 2.

ADR-002	Aplicar CQRS básico con Commands, Queries y Handlers sobre un único almacén
Título	Aplicar CQRS básico con Commands, Queries y Handlers sobre un único almacén
Estado	Aceptada
Contexto	El taller necesita cambiar el estado de la orden con reglas estrictas (registrar ingreso, autorizar cotización) y, por separado, consultar información para el Asesor de Servicio y el Cliente (estado de la orden, historial del Vehículo) y para la gerencia (tiempos promedio y órdenes con retraso, US-16). Reutilizar el modelo de escritura para lectura obligaría a cargar agregados completos solo para mostrar datos.
Alternativas	**Servicio CRUD único** que lee y escribe con el mismo modelo: simple, pero mezcla responsabilidades y expone el agregado a las vistas. **CQRS completo** con bases separadas y proyecciones por eventos: potente, pero su costo operativo es injustificado para un taller en esta etapa. **CQRS básico:** separación lógica de Commands y Queries con el mismo almacén de datos.
Decisión	Se adopta CQRS básico. Los Commands (RegistrarIngresoOrdenCommand, AutorizarCotizacionCommand) pasan por su Handler, el Unit of Work y la raíz del agregado. Las Queries (ConsultarEstadoOrdenQuery, ConsultarHistorialVehiculoQuery) devuelven DTO de lectura (DTOOrdenServicioDetalle, DTOHistorialServicio) y no cargan agregados. Los Application Services actúan como fachada del caso de uso y delegan al Handler. No se separan bases de datos.
Consecuencias	**Beneficios:** responsabilidades claras, DTO de lectura propios, camino de evolución hacia proyecciones si US-16 crece. **Costos aceptados:** más tipos (Command, Handler, DTO) y cierta duplicación entre modelos de lectura y escritura. **Parte sin separación:** la consulta de Cliente o Vehículo por identificador dentro de un Handler de escritura usa los repositorios del dominio, no una Query. **Trabajo posterior:** definir los Commands restantes (diagnóstico, fecha estimada de entrega, cobro).
Trazabilidad	CU-01, CU-03; US-01, US-04, US-16; RN-01, RN-13. Evidencia: tabla de Commands y Queries de la Etapa 6 y diagramas de secuencia 1 y 2.

ADR-003	Usar Repository por agregado con Unit of Work como frontera transaccional
Título	Usar Repository por agregado con Unit of Work como frontera transaccional
Estado	Aceptada
Contexto	El dominio no debe conocer SQL, ORM ni detalles de infraestructura. Las operaciones de caso de uso deben completarse juntas o no completarse: abrir una OrdenServicio y, si corresponde, registrar el Vehículo (CU-01) o recalcular la Cotización y guardar la orden (CU-03). Los Eventos de Dominio solo deben comunicarse si el cambio realmente se confirmó.
Alternativas	**Acceso directo a la base de datos desde los servicios:** acopla aplicación y dominio a la persistencia. **Repository sin Unit of Work:** cada guardar() confirma por separado; un fallo intermedio deja estados parciales. **Repository más Unit of Work:** interfaces expresadas en el lenguaje del dominio y una transacción por caso de uso.
Decisión	Se define una interfaz de Repository por raíz (OrdenServicioRepository) y por entidades con identidad propia referenciadas por identificador (ClienteRepository, VehiculoRepository, TecnicoRepository). Las interfaces viven en el dominio; sus implementaciones viven en infraestructura. El Unit of Work inicia al entrar al Handler (iniciar), confirma al final (confirmar), revierte ante cualquier excepción (revertir) y publica los Eventos de Dominio solo después del COMMIT.
Consecuencias	**Beneficios:** el dominio y la aplicación dependen de abstracciones (Dependency Inversion), pruebas con repositorios en memoria y rollback uniforme. **Costo aceptado:** CU-01 puede modificar dos agregados (Vehiculo y OrdenServicio) en la misma transacción cuando el VIN no existe; es una excepción consciente a «un agregado por transacción», justificada porque el Asesor de Servicio espera obtener la orden creada en una sola operación (supuesto S-01). **Riesgo:** concurrencia sobre una misma orden; se propone control de versión optimista (supuesto S-03). **Corrección registrada:** buscarPorId de OrdenServicioRepository devuelve una OrdenServicio y no una lista; se agrega buscarPorVin a VehiculoRepository.
Trazabilidad	CU-01 a CU-05; US-01, US-04, US-08, US-11; RN-01, RN-02, RN-09, RN-13. Evidencia: tabla de Repositories de la Etapa 5 y diagramas de secuencia 1 y 2.

ADR-004	Ubicar las reglas en el dominio y limitar el Application Service a la coordinación
Título	Ubicar las reglas en el dominio y limitar el Application Service a la coordinación
Estado	Aceptada
Contexto	Si las condiciones del negocio se acumulan en los Application Services, el dominio se convierte en simples contenedores de datos (modelo anémico) y las reglas se duplican entre casos de uso. Hay dos tipos de regla: invariantes de una orden (no modificar una orden Cerrada; no cobrar sin Cotización autorizada) y políticas que involucran varios conceptos (recálculo por rechazo parcial, créditos y descuentos de flotilla, US-17).
Alternativas	**Reglas en el Application Service o el Handler:** fácil de escribir, pero repetidas y difíciles de probar sin infraestructura. **Reglas en la entidad y el Value Object:** adecuadas para invariantes propias, pero inadecuadas para políticas que atraviesan varias entidades. **Reglas en la raíz del agregado y en Domain Services sin estado:** cada regla vive donde se protege.
Decisión	Las invariantes y transiciones de estado viven en OrdenServicio (raíz). Las políticas multi-entidad viven en los Domain Services CalcularCotizacion y ValidadorFlotilla, que no tienen estado ni dependencias de infraestructura. Los Application Services y Handlers solo reciben la solicitud, obtienen objetos, invocan el comportamiento del dominio, solicitan la persistencia y devuelven un resultado.
Consecuencias	**Beneficios:** pruebas de reglas sin base de datos, Application Services delgados y errores del dominio explicables con términos del negocio. **Costos aceptados:** la raíz recibe el Domain Service como parámetro, lo que añade un argumento a algunos métodos. **Riesgo:** que una regla nueva se escriba por comodidad en un Handler; se mitiga con la revisión SOLID y las pruebas de dominio. **Corrección de nombre:** el glosario escribía «ValidorFlotilla»; el nombre ubicuo es ValidadorFlotilla.
Trazabilidad	CU-03, US-04, US-17, US-08; RN-04, RN-13. Evidencia: columna «Reglas que delega» de la Etapa 4 y secuencia 2.